

Praevisa — Technical Briefing for the CTO

Date: 2026-06-08 · **Purpose:** orient a new technical lead on where the prediction engine actually stands — what is validated, what is promising-but-unproven, what data we depend on, and what decisions are open. Written to be self-contained; no prior project context assumed.

0. What Praevisa is

Praevisa forecasts how the **European Parliament (EP)** votes on legislation *before the vote happens*. Predictions are at the level of the eight-nine **political groups** (EPP, S&D, Renew, Greens, etc.), not individual MEPs and not just pass/fail. The intended buyers are public-affairs / lobbying / analyst teams who need to know which way a file is going and **who would have to change their vote to flip the outcome**.

1. 60-second primer on the process (the rest depends on it)

A legislative file under the **ordinary legislative procedure** (referenced as `YYYY/NNNN(COD)`) moves roughly:

*lead committee drafts and votes a report -> **plenary** (the full Parliament) votes at first reading -> **trilogue** negotiations with the Council of member states -> final adoption.*

Two stages produce public, per-group **roll-call** data we can use: the **committee** vote and the **plenary** vote. Both matter below.

The single most important fact about this domain: the vast majority of final plenary votes pass, and are *trivially* predictable from party arithmetic (a "always predict pass" rule is ~90% accurate). All real forecasting value lives in the small **contested** subset — close votes, rejected files, and files where a major group breaks from its usual position. Any honest evaluation has to be judged *there*, not on the easy majority.

2. Why this work was necessary (a credibility reset)

Before this session, the engine's reported quality rested on accuracy figures that, on inspection, had **no reproducible artifact** behind them — you could not regenerate them from code and data. The first job was therefore not modeling but **governance**: make every number reproducible from committed code, and delete anything that isn't. A CTO should read the rest of this memo through that lens — the bar we now hold is "one command regenerates it, or it doesn't get claimed."

3. What is now validated

3a. Reproducible baselines; unsubstantiated numbers removed

We rebuilt the baseline predictors as committed, one-command-reproducible code over a frozen 22-file test set of real EP votes (source: HowTheyVote.eu), and **independently re-derived the same numbers in a second repo with separate code and a fresh data pull**.

Terms, briefly:

- **Baseline A** — predict each group's vote as its own historical average ("what this group usually does").
- **Baseline C** — a fixed rule: the centrist coalition votes yes, the flanks vote no.
- **Floor / "pooled mean"** — predict the single overall average for every group; the dumbest reasonable predictor, and the one a real model must beat.
- **MSE** — mean squared error between predicted and actual group yes-rate (lower = better). Evaluated leave-one-out (each file predicted without seeing itself).

metric	previously cited (no artifact)	reproduced (two independent code paths)
Baseline A group MSE	0.1004	0.1656
Baseline C group MSE	0.1508	0.3240

The previously cited numbers do not reproduce under any faithful build; the real ones reproduce exactly, twice. Method and decisions are frozen in [METHODOLOGY.md](#) / [RESULTS.md](#).

3b. The hard finding: history-only baselines have no edge where it counts

Significance testing (cluster bootstrap, 10k resamples) shows **Baseline A is statistically indistinguishable from the pooled-mean floor** on the full set ($p = 0.35$). We then restricted evaluation to the **contested** subset — and it gets worse, not better: on no contested cut does Baseline A beat the floor, and on *rejected* files it is actively *worse* than the pooled mean.

Takeaway: "what a group usually does" is not a forecasting edge. This is a useful negative result — it tells us where *not* to invest — and it sets a hard, explicit bar: **a real model must beat the pooled-mean floor on the contested subset, prospectively.**

4. The first positive result — committee vote predicts plenary

Hypothesis: on contested files, a group's **committee-stage** vote (cast weeks before plenary, so legitimately a *pre-vote* signal) predicts how that group votes in plenary far better than any history-only baseline.

We built the data pipeline to test this (§5) and graded it time-split and pseudo-prospectively (the floors are fit only on data dated *before* each target; the committee vote precedes the plenary vote). Metric: per-group MSE, with paired bootstrap confidence intervals and a Wilcoxon test.

subset	n	committee signal MSE	floor MSE	95% CI of difference	p
all pairs	10	0.053	0.170	[-0.185, -0.047]	0.020
contested only	6	0.050	0.213	[-0.249, -0.056]	0.062

- The committee signal beats the floor on **9 of 10** files, a **~3-4× error reduction**, and the improvement is **statistically significant** on the full set ($p = 0.020$).
- On contested files the effect is *larger*; the confidence interval still excludes zero, but $n = 6$ is too small for the Wilcoxon test to reach $p < 0.05$ (a power limit, not a weak effect).
- **We stress-tested it.** A skeptic would say "committee and plenary are nearly the same vote." We split by stage: the genuinely-earlier **report-adoption** committee vote is the *strongest* predictor, while the late, near-identical "provisional agreement" vote is *weaker* — the opposite of what an artifact would show. The signal is real.

This is the first evidence in the project that any pre-vote signal beats the established baseline on the votes that matter. It is not yet a validated track record (see §7).

5. Data sources and their reliability (important for a CTO)

- **HowTheyVote.eu** — our source for plenary roll-calls (per-group results). Two access paths: a JSON API and a nightly **bulk CSV export**. **Gotcha we hit and fixed:** the API's *list* endpoint is a capped/rolling window (~2,352 of ~24,000 votes) that silently omits older votes — it cannot be used to enumerate or to find a given file's vote. We now resolve files via the **complete bulk export** and fetch details by id.
- **Committee roll-calls** — there is **no API**; each committee publishes per-meeting results as **PDFs** on europarl.europa.eu, and that page is a **rolling window** (old PDFs age off). Implications: (a) we parse

PDFs (brittle by nature; handled defensively), and (b) **the corpus must accumulate over time** — a one-shot scrape cannot rebuild history. The scraper now *merges* rather than overwrites, and the corpora are committed (not treated as regenerable).

- **Reliability summary:** plenary data is solid and complete via the bulk export; committee data is the fragile, accumulate-over-time dependency and the main data risk.

6. What you are inheriting (codebase state)

- **Language/stack:** Python; reproducible via `uv`; results are committed JSON + Markdown, regenerated by one command each. Statistics use numpy/scipy. No heavyweight ML framework in the validated path — the baselines and the committee signal are deliberately simple.
- **Validated vs experimental:** the §3 baselines and §4 Stage-A result are committed and reproducible. The forward-prediction product engine (an LLM/Monte-Carlo pipeline that predates this work) is **not** part of what's validated here and should be treated as experimental.
- **Automation just stood up:** a **Stage B** pre-commitment harness writes immutable, hash-stamped predictions for files that have a committee vote but no plenary vote yet, and grades them only once the plenary vote exists *and is dated after the commit*. A weekly macOS **LaunchAgent** runs the full loop (scrape -> grade -> predict -> commit); its first run completed end-to-end. The git commit timestamp is the pre-commitment proof.
- **Caveat to fix for external defensibility:** the repo has **no git remote**, so pre-commitment is currently proven only by *local* history. For an investor- or court-defensible track record, add a remote and push at prediction time.

7. Honest limitations and risks

- **Sample is tiny.** Stage A is 10 files (6 contested), one snapshot; the cleanest contested evidence is ~2 effectively-independent files. Encouraging, not conclusive.
- **Not yet truly prospective.** Stage A used historical data with a time-split. The real test (Stage B, blinded pre-commitment) needs **~2-6 months** to accumulate contested predictions — right now there are *zero* contested files awaiting a plenary vote, because the ones we have already voted.
- **Narrow scope.** The committee signal buys ~weeks of lead time on files that have reached committee. It does **not** forecast far ahead, and it does **not** cover the **Council/trilogue** dimension, where much of the hypothesized commercial value lives and where *nothing is built yet*.
- **Competition.** At least one better-resourced competitor already does forward EP prediction at ~80% accuracy on millions of historical votes. Our only defensible ground is the contested/upstream niche this signal targets — *if* it validates prospectively.

8. Recommendation and where your judgment is needed

- 1 **Do not make external/marketing/investor claims off this yet.** It is one promising snapshot. Let the automated weekly cadence build a *prospective, blinded* contested track record over the next couple of months; then we either have a defensible edge or we learn the snapshot was lucky — both worth more than asserting it now.
- 2 **Decisions for you:** (a) is the Council/trilogue layer worth real investment, given it's where value lives but data is hardest? (b) add a git remote so pre-commitments are externally provable? (c) how much engineering to harden the committee PDF scraper, our main data fragility?

9. Get oriented (run these from the `praevisa` repo)

```
uv run python -m praevisa.baseline_eval      # baselines – the real, reproducible numbers
uv run python -m praevisa.baseline_robustness # significance: Baseline A ties the floor
uv run python -m praevisa.contested_subset    # contested subset indicts Baseline A
uv run python -m praevisa.stage0_feasibility  # data pairs available: 13 (6 contested)
uv run python -m praevisa.stage_a            # committee signal beats the floor (n=10)
uv run python -m praevisa.stage_b status      # prospective prediction ledger
```

Start with `METHODOLOGY.md` and `RESULTS.md` for full method and numbers. The work in this briefing is the commit range from `dcf442b` (significance testing) through `12c5e35` (this memo); notable commits: `2dcf53d` (data resolver), `caaddfc` (data feasibility), `0fea9ca` (Stage A result), `649796b` (Stage B harness), `7805292/39ff209` (automation).